e.6

```
# Import libraries

import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder

from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier

from sklearn.metrics import accuracy_score



# Load dataset

data = pd.read_csv("Employee.csv")


# Convert categorical data

le = LabelEncoder()

for col in data.select_dtypes(include='object'):
    data[col] = le.fit_transform(data[col])


# Split input and output

X = data.drop("LeaveOrNot", axis=1)

y = data["LeaveOrNot"]



# Split dataset

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)



# -------------------------
# Random Forest
# -------------------------

rf = RandomForestClassifier(
    n_estimators=100
)


rf.fit(
    X_train,
    y_train
)


rf_prediction = rf.predict(
    X_test
)
```

123 High Street, Anytown, County, Postcode 01234 567 890
no_reply@example.com

```


rf_accuracy = accuracy_score(
    y_test,
```

```
    rf_prediction
)



# -------------------------
# Gradient Boosting
# -------------------------

gb = GradientBoostingClassifier()


gb.fit(
    X_train,
    y_train
)


gb_prediction = gb.predict(
    X_test
)


gb_accuracy = accuracy_score(
    y_test,
    gb_prediction
)



# Display results

print("Random Forest Accuracy:",
      rf_accuracy)


print("Gradient Boosting Accuracy:",
      gb_accuracy)

e.5
# Import libraries

import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score


# Load dataset

data = pd.read_csv("Employee.csv")


# Display dataset

print(data.head())


# Convert categorical columns

le = LabelEncoder()

for col in data.select_dtypes(include='object'):
    data[col] = le.fit_transform(data[col])


# Split input and output

X = data.drop("LeaveOrNot", axis=1)
```

```
y = data["LeaveOrNot"]


# Train test split

X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)


# Feature scaling (needed for SVM and KNN)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)



# --------------------
# Decision Tree
# --------------------

dt = DecisionTreeClassifier()

dt.fit(X_train, y_train)

dt_prediction = dt.predict(X_test)

dt_accuracy = accuracy_score(
    y_test,
    dt_prediction
)



# --------------------
# SVM
# --------------------

svm = SVC()

svm.fit(X_train, y_train)

svm_prediction = svm.predict(X_test)

svm_accuracy = accuracy_score(
    y_test,
    svm_prediction
)



# --------------------
# KNN
# --------------------

knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train, y_train)

knn_prediction = knn.predict(X_test)

knn_accuracy = accuracy_score(
    y_test,
    knn_prediction
)



# Results

print("Decision Tree Accuracy:",
```

```
    dt_accuracy)

print("SVM Accuracy:",
    svm_accuracy)

print("KNN Accuracy:",
    knn_accuracy)
```

e.4

```
# Import libraries
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, LogisticRegression
from sklearn.metrics import mean_squared_error, r2_score, accuracy_score, confusion_matrix

# Load dataset
data = pd.read_csv("data.csv")

print(data.head())


# ---------- Linear Regression ----------

# Features and target
X = data[['Feature']]
y = data['Target']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Linear Regression model
linear_model = LinearRegression()

# Train model
linear_model.fit(X_train, y_train)

# Prediction
y_pred = linear_model.predict(X_test)

# Evaluation
print("Linear Regression")
print("MSE:", mean_squared_error(y_test, y_pred))
print("R2 Score:", r2_score(y_test, y_pred))


# ---------- Logistic Regression ----------

# Features and class
X = data[['Feature1','Feature2']]
y = data['Class']

# Split data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Create Logistic Regression model
logistic_model = LogisticRegression()

# Train model
logistic_model.fit(X_train, y_train)

# Prediction
y_pred = logistic_model.predict(X_test)

# Evaluation
print("\nLogistic Regression")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:")
print(confusion_matrix(y_test, y_pred))
```

e.3

```
# Import libraries
import pandas as pd
```

```
import matplotlib.pyplot as plt
import seaborn as sns

# Load dataset
df = pd.read_csv("Employee.csv")

# Display data
print(df.head())

# Dataset information
df.info()

# Statistical analysis
print(df.describe())

# Check missing values
print(df.isnull().sum())

# Data distribution using histogram
df.hist(figsize=(10,8))
plt.show()

# Detect outliers using boxplot
sns.boxplot(data=df)
plt.xticks(rotation=90)
plt.show()

# Categorical data visualization
sns.countplot(x='Department', data=df)
plt.show()

# Relationship between features
sns.scatterplot(x='Age', y='Salary', data=df)
plt.show()

# Correlation heatmap
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True)
plt.show()

# Pairwise relationship
sns.pairplot(df)
plt.show() <This message was edited>

e.2
# Import libraries
import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.feature_selection import SelectKBest, f_classif

# Load dataset
df = pd.read_csv("Employee.csv")

# Display dataset
print(df.head())

# Convert categorical columns into numerical
le = LabelEncoder()

for col in df.select_dtypes(include='object'):
    df[col] = le.fit_transform(df[col])

# Feature Engineering
df["New_Feature"] = df["Experience"] / df["Age"]

print(df.head())

# Split data
X = df.drop("Salary", axis=1)
y = df["Salary"]

# Feature Selection
selector = SelectKBest(score_func=f_classif, k=3)

X_selected = selector.fit_transform(X, y)
```

```
# Display selected features
selected = X.columns[selector.get_support()]

print("Selected Features:")
print(selected)
```

e.1

```
import pandas as pd
from sklearn.preprocessing import LabelEncoder, MinMaxScaler

# Load data
data = pd.read_csv("Employee.csv")

# Handle missing values
data.fillna(data.mean(numeric_only=True), inplace=True)

# Object to numerical
le = LabelEncoder()
for i in data.select_dtypes('object'):
    data[i] = le.fit_transform(data[i])

# Normalization
scaler = MinMaxScaler()
data[data.columns] = scaler.fit_transform(data)

# Display data
data.head() <This message was edited>
```